APACHE MAVEN — COMPLETE STUDY NOTES

Azure DevOps Track | Interview Ready | Code + Explanation
Cognizant Fresher Programme | 16 Topics Covered

TOPIC 1 — WHAT IS MAVEN & WHY IT EXISTS

The Problem Before Maven

- **Before Maven: ClickOps for dependencies** — manually downloading every JAR file a project needed
- **Problems with manual approach:**
  - Slow and error-prone
  - Cannot reproduce the same dependency set exactly across machines
  - Version conflicts across team members
  - No way to track what changed and when
- **Maven solves this by declaring what you need and letting the tool manage everything else**

What Maven Does

- **Dependency Management:** declare what you need, Maven downloads it and all transitive dependencies
- **Standard Build Process:** fixed sequence from source code to deployable artifact — same on every machine
- **Convention over Configuration:** follow the standard structure, need almost zero configuration
- **Plugin System:** Maven is an engine that calls plugins — nearly all functionality is plugin-provided

The One Command Reality

```
mvn install
```

This single command compiles source, runs tests, packages into a JAR, and installs to the local repository — downloading any missing dependencies automatically.

> ⚠ **NOTE:** Maven has become the de facto standard build tool for Java. Any software developer working with Java needs to understand it.

INTERVIEW Q&A

Q: What is Maven?
A: A build automation and dependency management tool that enforces project structure through convention, manages the full build lifecycle, and uses a plugin-based architecture where nearly all functionality is provided by plugins.

Q: Why use Maven over manually managing JARs?
A: Automatic transitive dependency resolution, a standardized reproducible build lifecycle across all machines, and access to Maven Central — a massive public repository of open source libraries.

TOPIC 2 — MAVEN COORDINATES

What Are Maven Coordinates

- **Every artifact in Maven is identified by three values called Maven Coordinates**
- **Together they are** guaranteed to be unique across the entire Maven ecosystem
- **Used in three places:** your project identity, declaring dependencies, configuring plugins

The Three Parts

```
<groupId>org.apache.commons</groupId>
<artifactId>commons-lang3</artifactId>
<version>3.4</version>
```

`<groupId>` → the organization that owns the artifact — follows reverse domain convention: com.cognizant, org.apache.commons

`<artifactId>` → the name of the specific project or library: commons-lang3, maven-quick-start

`<version>` → which version to fetch: 3.4, 1.0, 4.12 — Maven cannot resolve without this

Local Repository Path Pattern (MEMORISE THIS)

```
Format: ~/.m2/repository/groupId(dots-as-slashes)/artifactId/version/artifact.jar
Example: ~/.m2/repository/org/apache/commons/commons-lang3/3.4/commons-lang3-3.4.jar
```

Three Places Coordinates Appear

- **1. Your project identity:** at the top of your pom.xml — who you are
- **2. Declaring dependencies:** inside <dependencies> — libraries you need
- **3. Configuring plugins:** inside <build><plugins> — which plugin to use

INTERVIEW Q&A

Q: What are Maven Coordinates?
A: A combination of groupId, artifactId, and version that uniquely identifies any artifact in the Maven ecosystem. groupId is the organization (reverse domain), artifactId is the project name, version is the specific release.

Q: What is the .m2 repository folder structure?
A: Maven mirrors the coordinates as a directory path: groupId (dots replaced by slashes) / artifactId / version / artifact.jar and artifact.pom.

TOPIC 3 — THE POM FILE

What is the POM

- **POM = Project Object Model**
- **XML file named** pom.xml at the root of every Maven project
- **The single source of truth** for everything Maven needs — coordinates, dependencies, plugins, build config
- **First thing Maven reads** when any command is run

The Minimal POM

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>clinic.programming</groupId>
    <artifactId>maven-quick-start</artifactId>
    <version>1.0</version>
</project>
```

`<project>` → opening tag with namespace — pure boilerplate, copy as-is into every project

`<modelVersion>4.0.0</modelVersion>` → always 4.0.0 — tells Maven which POM schema version you are using

`<groupId>` → your organization identifier in reverse domain format

`<artifactId>` → the name of this specific project/artifact

`<version>` → the version number of your artifact

Key POM Sections

- **<packaging>:** controls artifact type. jar (default), war, pom, ear. Omit for JAR.
- **<properties>:** variables reusable inside the POM using ${propertyName} syntax
- **<dependencies>:** list of all external libraries your project needs
- **<build>:** configure plugins and override default build behaviour

⚠ **NOTE:** POM supports inheritance. Every pom.xml automatically inherits from the Super POM — Maven's built-in master POM that provides all defaults.

INTERVIEW Q&A

Q: What is a POM file?
A: The Project Object Model — an XML file named pom.xml at the root of every Maven project. It defines coordinates, dependencies, build configuration, and plugins. It is Maven's single source of truth.

Q: What is the minimum required in a pom.xml?
A: modelVersion (always 4.0.0), groupId, artifactId, and version. Everything else is optional or inherited from the Super POM.

Q: What is <packaging> in Maven?
A: Controls the artifact type Maven produces. Default is jar. Other values: war, pom, ear.

TOPIC 4 — SUPER POM & CONVENTION OVER CONFIGURATION

What is the Super POM

- **Maven's built-in master POM that every project automatically inherits from**
- **You never see it or write it** — it is always silently present
- **Lives inside Maven's own JAR files**
- **Think of it like a base class in Java** — your pom.xml is the subclass

What the Super POM Provides

Default Setting	Value
Source directory	src/main/java
Test directory	src/test/java
Output directory	target/
Remote repository	https://repo.maven.apache.org/maven2
Default plugin versions	Compiler, Surefire, JAR, Install, Deploy plugins
Final artifact name	artifactId-version.jar

Convention over Configuration

- **If you follow Maven's agreed conventions** — you need almost zero configuration
- **You only write configuration** when you want to deviate from the convention
- **Only specify options that are different** for your specific project

The Effective POM

- **The fully merged view:** Super POM + parent POMs + your own POM combined
- **Shows every setting actually in effect** for your build

```
mvn help:effective-pom
```

Also visible as a dedicated tab in Eclipse's POM editor.

> ⚠ **NOTE:** The Effective POM is extremely useful when debugging why a build is behaving unexpectedly — it shows every default setting that Maven has silently applied from the Super POM.

INTERVIEW Q&A

Q: What is the Super POM?
A: Maven's built-in master POM that every project automatically inherits from. It defines source directories, output directory, plugin versions, and the central repository URL — enabling Convention over Configuration.

Q: What is Convention over Configuration in Maven?
A: Maven provides sensible defaults so that if you follow the standard project structure, you need almost no configuration to get a working build. You only configure what differs from convention.

Q: What is the Effective POM?

A: The fully merged POM combining Super POM + any parent POMs + your own pom.xml. Shows every setting actually in effect. Viewed with mvn help:effective-pom or the Eclipse POM editor Effective POM tab.

TOPIC 5 — STANDARD DIRECTORY STRUCTURE

The Complete Standard Layout

```
my-project/
├── pom.xml                    ← Maven project definition file
├── src/
│   ├── main/
│   │   ├── java/              ← Production Java source code
│   │   ├── resources/         ← Config files, properties, XML
│   │   └── webapp/            ← Web files (WAR projects only)
│   └── test/
│       ├── java/              ← Unit test source code
│       └── resources/         ← Test-only config files
└── target/                    ← ALL build output (auto-generated)
    ├── classes/               ← Compiled .class files
    ├── test-classes/          ← Compiled test .class files
    ├── surefire-reports/      ← Unit test result reports
    └── my-project-1.0.jar     ← Final packaged artifact
```

What Each Folder Is For

- **src/main/java:** all production Java source that ends up in the final JAR
- **src/main/resources:** non-Java files needed at runtime — properties, XML configs. Maven copies these to classpath root.
- **src/test/java:** unit test classes. Never bundled into production artifact.
- **src/test/resources:** config files needed only during testing
- **target/:** Maven's output directory. Auto-generated. Always in .gitignore. Safe to delete entirely.

⚠ **NOTE:** .gitignore must always contain target/ — it is auto-generated on every build and should never be committed to source control.

INTERVIEW Q&A

Q: What is Maven's standard directory structure?
A: Source in src/main/java, resources in src/main/resources, tests in src/test/java, test resources in src/test/resources, build output in target/. pom.xml at project root.

Q: What is the target/ directory?
A: Maven's output directory where compiled classes, test results, and final artifacts go. Auto-generated — always in .gitignore. Safe to delete; Maven regenerates it on next build.

Q: Why follow Maven's standard directory structure?
A: Enables Convention over Configuration — every tool, CI system, and developer that understands Maven can work with your project immediately without extra documentation.

TOPIC 6 — MAVEN LIFECYCLES, PHASES & GOALS

Three Levels: Goal → Phase → Lifecycle

- **Goal:** smallest unit of work, provided by a plugin, named as pluginName:goalName
- **Phase:** a named step in a lifecycle — when called, ALL preceding phases run first
- **Lifecycle:** a complete ordered sequence of phases

Goals — Examples

`compiler:compile` → the compile goal from the compiler plugin — compiles Java source
`surefire:test` → the test goal from surefire — runs unit tests
`jar:jar` → the jar goal from the jar plugin — packages into JAR
`dependency:tree` → prints the full dependency tree

The Default Lifecycle — Phases in Order

Phase	What it does
validate	Validates pom.xml is correct
compile	Compiles src/main/java → target/classes
test-compile	Compiles src/test/java → target/test-classes
test	Runs unit tests via Surefire plugin
package	Packages into JAR/WAR in target/
verify	Verifies package meets quality criteria
install	Copies artifact to ~/.m2/repository
deploy	Pushes artifact to remote repository (Nexus/Artifactory)

Clean Lifecycle

- **Completely separate lifecycle** from the default lifecycle
- **The clean phase** deletes the entire target/ directory
- **Must be specified explicitly** alongside default lifecycle phases

```
mvn clean install   ← clean lifecycle + default lifecycle, both run
```

⚠ **NOTE:** When you call mvn package, Maven does NOT just run package — it runs validate, compile, test-compile, test first, then package. All preceding phases always execute.

INTERVIEW Q&A

Q: What is the difference between a Goal, Phase, and Lifecycle?
A: A Goal is the smallest unit of work provided by a plugin (e.g. compiler:compile). A Phase is a named step in a lifecycle with goals bound to it. A Lifecycle is an ordered sequence of phases. Calling a phase runs ALL preceding phases first.

Q: What happens when you run mvn package?

A: Maven runs every phase before package first — validate, compile, test-compile, test — then runs package, producing a JAR in target/.

Q: Why use mvn clean install instead of just mvn install?
A: clean deletes target/ first, ensuring a fresh build with no leftover artifacts. Since clean belongs to a separate lifecycle, it must be specified explicitly.

Q: How many lifecycles does Maven have?
A: Three: Default (build and deploy), Clean (delete build output), Site (generate documentation).

TOPIC 7 — CORE CLI COMMANDS

Command Reference

Command	What it does	To .m2?
mvn compile	Compiles src/main/java only	No
mvn test	Compiles + runs all unit tests	No
mvn package	Compiles + tests + creates JAR in target/	No
mvn install	Everything in package + copies artifact to .m2	Yes
mvn clean	Deletes entire target/ directory	No
mvn clean install	Fresh build all the way to .m2 (most common)	Yes
mvn clean package	Fresh build, JAR in target/ only	No
mvn deploy	Full build + pushes to remote Nexus/Artifactory	Yes + Remote
mvn dependency:tree	Prints full dependency tree — no build triggered	No

Useful Flags

- `-DskipTests` → skip test execution but still compile test classes
- `-Dmaven.test.skip=true` → skip both compilation AND execution of tests
- `-o` → offline mode — only use what is in local .m2 cache
- `-X` → debug mode — verbose output showing every internal step Maven takes
- `-q` → quiet mode — suppress most output, show errors only

⚠ **NOTE:** In CI/CD pipelines (Jenkins, Azure DevOps, GitHub Actions), tests should NEVER be skipped. The -DskipTests flag is only for exceptional local circumstances.

INTERVIEW Q&A

Q: What is the difference between mvn package and mvn install?
A: package produces the artifact in target/ but does not place it elsewhere. install does everything package does and copies the artifact to ~/.m2/repository, making it available to other local Maven projects.

Q: What does mvn clean install do step by step?
A: Deletes target/ (clean lifecycle), then validates POM, compiles source, compiles tests, runs tests, packages JAR, copies JAR to ~/.m2 (default lifecycle up to install).

Q: What is the -X flag?
A: Enables debug mode — verbose output showing every internal step Maven takes. Used for diagnosing build issues.

TOPIC 8 — PLUGINS

What Are Plugins

- **Every Goal in Maven is provided by a Plugin**
- **Maven itself is an engine** that calls plugins — without plugins Maven does nothing
- **Plugins are artifacts** stored in Maven repositories and downloaded just like dependencies
- **Two categories:** built-in core plugins and additional plugins

Core Plugins Reference

Plugin	Triggered by
maven-compiler-plugin	compile phase
maven-surefire-plugin	test phase
maven-jar-plugin	package phase
maven-install-plugin	install phase
maven-deploy-plugin	deploy phase
maven-clean-plugin	clean phase

Plugin Configuration Syntax

```
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.8.1</version>
            <configuration>
                <source>1.8</source>
                <target>1.8</target>
            </configuration>
        </plugin>
    </plugins>
</build>
```

`<build>` → anything that modifies the build process lives in this section

`<plugins>` → collection of plugin elements to modify or configure

`groupId + artifactId + version` → Maven Coordinates identifying exactly which plugin to use

`<configuration>` → block containing the specific settings to override for this plugin

Calling a Plugin Goal Directly

```
mvn dependency:tree
mvn archetype:generate
mvn help:effective-pom
```

Use pluginName:goalName to bypass the lifecycle and run one specific task directly.

INTERVIEW Q&A

Q: What are Maven plugins?

A: Components that provide all actual functionality in Maven. Every goal is provided by a plugin. Maven itself orchestrates plugin execution. Plugins are artifacts stored in Maven repositories.

Q: How do you configure a plugin in Maven?
A: Inside <build><plugins> in pom.xml, using a <plugin> block with Maven Coordinates and a <configuration> section containing the specific settings to override.

Q: Why explicitly specify plugin versions?
A: To ensure build reproducibility. Without an explicit version, Maven uses the Super POM default, which can change across Maven releases and break your build.

TOPIC 9 — COMPILER PLUGIN & JAVA VERSION

The Default Problem

- **Maven compiles targeting Java 1.5 by default** regardless of which JDK is installed
- **Writing modern Java code without configuring this plugin causes build failure:**

```
diamond operator is not supported in -source 1.5
(use -source 7 or higher to enable diamond operator)
```

Method 1 — Plugin Configuration Block (Explicit)

```
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.8.1</version>
            <configuration>
                <source>1.8</source>
                <target>1.8</target>
            </configuration>
        </plugin>
    </plugins>
</build>
```

Method 2 — Properties (Preferred for simplicity)

```
<properties>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
</properties>
```

Achieves identical result with less code. The compiler plugin reads these property names automatically. No <build> section needed.

Method 3 — Release Parameter (Java 9+)

```
<properties>
    <maven.compiler.release>11</maven.compiler.release>
</properties>
```

Combines source, target, and API-level enforcement in one parameter. Recommended for Java 9+ projects.

source vs target — Key Difference

`source` → which Java language syntax version is ALLOWED in your source code

`target` → which JVM version the compiled .class files must be COMPATIBLE with

They are almost always set to the same value.

Java Version Naming Reference

Java Name	Use in Maven
Java 8 / JDK 1.8	1.8 or 8 (both work)
Java 11	11
Java 17	17

Java 21	21

⚠ **NOTE:** Java 8 is also called JDK 1.8 — Oracle's confusing naming. 1.8 and 8 are identical in Maven. From Java 9 onwards, just use the number directly.

INTERVIEW Q&A

Q: What is the default Java version Maven compiles against?
A: Java 1.5, regardless of the JDK installed. Set by the Super POM's default compiler plugin configuration.

Q: What is the difference between source and target?
A: source defines which Java language syntax is permitted in your source code. target defines which JVM version the compiled bytecode is compatible with. Almost always set to the same value.

Q: What is the release parameter?
A: Introduced in Java 9, it combines source, target, and API-level enforcement in one parameter, preventing accidental use of APIs unavailable in the target version.

TOPIC 10 — DEPENDENCY MANAGEMENT

Declaring a Dependency

```
<dependencies>
    <dependency>
        <groupId>org.apache.commons</groupId>
        <artifactId>commons-lang3</artifactId>
        <version>3.4</version>
    </dependency>
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.12</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

How Maven Resolves Dependencies

- **Step 1 — Check local first:** Maven checks ~/.m2/repository. If found, uses it immediately — no internet needed.
- **Step 2 — Remote download:** if not local, downloads from Maven Central and stores in .m2
- **Step 3 — Transitive resolution:** reads the downloaded artifact's pom.xml, downloads its dependencies recursively

Transitive Dependencies

- **A transitive dependency is** a dependency of your dependency
- **You declare JUnit 4.12** — JUnit depends on hamcrest-core — Maven downloads both automatically
- **Maven resolves the entire dependency tree** recursively — you only declare your direct dependencies

```
clinic.programming:maven-quick-start:jar:1.0
+- org.apache.commons:commons-lang3:jar:3.4:compile
\- junit:junit:jar:4.12:test
   \- org.hamcrest:hamcrest-core:jar:1.1:test   ← transitive
```

Finding Dependencies

- **Maven Central at** search.maven.org — search for the library, copy the XML snippet into pom.xml

INTERVIEW Q&A

Q: How does Maven handle dependency management?
A: You declare direct dependencies in <dependencies> using Maven Coordinates. Maven checks local .m2 first, downloads from Maven Central if needed, and recursively resolves all transitive dependencies automatically.

Q: What is a transitive dependency?
A: A dependency of one of your declared dependencies. Maven resolves and downloads the entire dependency tree automatically — you only declare direct dependencies.

Q: Where do you find dependencies to add?

A: Maven Central at search.maven.org — provides the exact <dependency> XML to copy into your POM.

TOPIC 11 — DEPENDENCY SCOPES

What is Scope

- **Scope controls:** when a dependency is available, whether it is in the final artifact, whether it is passed transitively
- **Default scope is** compile — if you do not specify anything

All Six Scopes

compile — Default

- **Available at:** compile time, test time, runtime
- **In final artifact:** Yes
- **Transitive:** Yes
- **Example:** commons-lang3, any utility library your code uses directly

test — Testing Only

- **Available at:** test compilation and execution only
- **In final artifact:** No
- **Transitive:** No
- **Example:** JUnit, Mockito, AssertJ — any library purely for writing and running tests

provided — Compile but Not Bundled

- **Available at:** compile time and test time
- **In final artifact:** No — runtime container provides it
- **Example:** Servlet API — Tomcat already ships with it, bundling it causes conflicts

runtime — Runtime but Not Compile

- **Available at:** test time and runtime
- **In final artifact:** Yes
- **Example:** JDBC driver — your code compiles against java.sql interfaces, driver is only needed at runtime

system — Local File (Avoid)

- **Points to a JAR on local filesystem** using <systemPath>
- **Avoid — breaks build portability:** other developers won't have the file at that exact path

import — BOM Imports Only

- **Only inside <dependencyManagement>** — imports versions from a BOM POM
- **Example:** Spring Boot parent POM

Scope Comparison Table (MEMORISE THIS)

Scope	In Final Artifact?	Transitive?
compile	Yes	Yes

test	No	No
provided	No	No
runtime	Yes	Yes
system	No	No
import	N/A	N/A

INTERVIEW Q&A

Q: What are Maven dependency scopes?
A: Scope controls when a dependency is available, whether it is in the final artifact, and whether it is passed transitively. Six scopes: compile (default), test, provided, runtime, system, import.

Q: Difference between provided and runtime scope?
A: provided is available at compile time but not in the final artifact — the container provides it at runtime (e.g. Servlet API). runtime is not available at compile time but is in the final artifact — needed when the app runs (e.g. JDBC driver).

Q: Why use test scope?
A: Ensures testing libraries like JUnit are only available during test compilation and execution and are never bundled into the production artifact.

TOPIC 12 — MAVEN REPOSITORIES

Three Types of Repositories

1. Local Repository (~/.m2/repository)

- **A cache folder on your own machine**
- **Maven always checks here first** before going to the internet
- **Safe to delete entirely** — Maven re-downloads everything it needs on next build

```
~/.m2/repository/org/apache/commons/commons-lang3/3.4/
    commons-lang3-3.4.jar
    commons-lang3-3.4.pom    ← Maven metadata
    commons-lang3-3.4.jar.sha1 ← checksum to verify download integrity
```

2. Maven Central

- **The default public remote repository** built into Maven via the Super POM
- **URL:** https://repo.maven.apache.org/maven2
- **Search at:** search.maven.org
- **Searched automatically** when a dependency is not found locally

3. Private Remote Repositories (Nexus / Artifactory)

- **Used in enterprise environments** like Cognizant
- **Proxy/Cache:** sits between developers and Maven Central — faster builds, offline resilience
- **Hosts internal artifacts:** company-developed libraries other teams depend on
- **Security/Governance:** controls which external libraries are allowed, blocks vulnerable versions

Dependency Resolution Order (MEMORISE THIS)

```
1. Check ~/.m2/repository (local)  →  2. Check Nexus/Artifactory (if
configured)  →  3. Check Maven Central
```

Adding a Repository to pom.xml

```
<repositories>
    <repository>
        <id>jboss-public</id>
        <url>https://repository.jboss.org/nexus/content/groups/public</url>
    </repository>
</repositories>
```

INTERVIEW Q&A

Q: What are the types of Maven repositories?
A: Three: local (~/.m2/repository) caches everything on your machine, Maven Central is the default public remote repository, private repositories like Nexus or Artifactory used in enterprise environments.

Q: What is Maven Central?
A: The default public remote repository at repo.maven.apache.org, containing virtually all open source Java libraries. Built into Maven via the Super POM.

Q: What is Nexus or Artifactory?
A: Private artifact repository managers used in enterprises. They proxy Maven Central, host internally developed artifacts, and enforce security policies on dependencies.

Q: What order does Maven follow when resolving?
A: Local repository first, then any configured private/additional repositories, then Maven Central.

TOPIC 13 — UNIT TESTING WITH MAVEN

Maven's Approach to Testing

- **Testing is non-negotiable:** if tests fail, the build fails — you cannot produce a JAR until all tests pass
- **Test source lives in:** src/test/java — completely separate from production code
- **JUnit is declared as test scope** so it is never bundled into the production artifact

JUnit Dependency Declaration

```
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.12</version>
    <scope>test</scope>
</dependency>
```

The Surefire Plugin

- **maven-surefire-plugin** runs unit tests during the test phase
- **Built-in plugin** — no declaration needed unless changing its behaviour
- **Automatically discovers test classes named with:**
  - Suffix Test (e.g. ApplicationTest)
  - Prefix Test (e.g. TestApplication)
  - Suffix TestCase

Running Tests — Commands

Command	What it does
mvn test	Compile + run all tests only
mvn clean install	Full build including tests
mvn install -DskipTests	Build but skip test execution
mvn install -Dmaven.test.skip=true	Build, skip compile AND execution of tests

Surefire Reports — target/surefire-reports/

- **.txt file:** human-readable summary — tests run, failures, errors, skipped
- **.xml file:** machine-readable results consumed by CI/CD systems and tools like SonarQube

Test Phase in the Build Flow

```
clean → compile → test-compile → test → package → install
                                  ↑
                    Surefire runs here
                    Generates target/surefire-reports/
                    Any test fail = BUILD FAILURE = stop
```

⚠ **NOTE:** In CI/CD pipelines, tests should never be skipped. The -DskipTests flag is only for exceptional local debugging circumstances.

INTERVIEW Q&A

Q: How does Maven handle unit testing?
A: Maven has a dedicated test phase using the Surefire plugin. If any test fails, the build fails. Tests go in src/test/java. JUnit is declared test scope so it is never in the production artifact.

Q: What is the Maven Surefire plugin?
A: Built-in plugin that discovers and runs unit tests during the test phase. Finds classes with Test suffix/prefix. Generates reports in target/surefire-reports/ in .txt (human-readable) and .xml (machine-readable) formats.

Q: What is in target/surefire-reports/?
A: Two files per test class: a .txt with human-readable summary of tests run/passed/failed/skipped, and an .xml for CI/CD systems and tools like SonarQube.

TOPIC 14 — DEPENDENCY:TREE

What is dependency:tree

- **Prints the complete dependency graph** — all direct and transitive dependencies, versions, and scopes
- **A direct goal call** — does NOT trigger the build lifecycle
- **Provided by the Maven Dependency Plugin**

```
mvn dependency:tree
```

Reading the Output

```
clinic.programming:maven-quick-start:jar:1.0
+- org.apache.commons:commons-lang3:jar:3.4:compile
\- junit:junit:jar:4.12:test
   \- org.hamcrest:hamcrest-core:jar:1.1:test
```

Output Format (MEMORISE THIS)

```
Format: groupId:artifactId:packaging:version:scope
+- more items follow at this level
\- last item at this level
indented entry= transitive dependency pulled in by the entry above it
```

Why It Is Useful

- **See all transitive dependencies:** large frameworks like Spring can pull in 50+ you never declared
- **Diagnose version conflicts:** see exactly which library is pulling in which version
- **Verify scopes:** confirm every dependency has the correct scope

Saving Output to File

```
mvn dependency:tree > dependency-list.txt
```

Other Dependency Plugin Goals

Goal	Purpose
dependency:tree	Full dependency tree with scopes
dependency:resolve	Flat list of all resolved dependencies
dependency:copy-dependencies	Copies all JARs to a specified folder
dependency:analyze	Reports unused declared and used undeclared deps

INTERVIEW Q&A

Q: What does mvn dependency:tree do?
A: Prints the complete dependency tree showing all direct and transitive dependencies, their versions, and scopes. A direct goal call — does not trigger the build lifecycle.

Q: Why would you use dependency:tree?

A: To see all dependencies including transitive ones, diagnose version conflicts, and verify every dependency has the correct scope.

Q: What is the format of each entry in dependency:tree output?
A: groupId:artifactId:packaging:version:scope. Indented entries are transitive dependencies pulled in by the entry above them.

TOPIC 15 — MAVEN ARCHETYPES

What is a Maven Archetype

- **A project template** — generates a complete project skeleton in seconds
- **Instead of manually creating folders and files,** one command does everything
- **Provided by the Maven Archetype Plugin** — one of the default plugins
- **After generation,** the project has folder structure, starter pom.xml, and sample source code

Interactive Mode

```
mvn archetype:generate
```

Launches interactive mode. Steps:

- **Filter archetype list:** type maven-archetype to narrow from 1500+ archetypes
- **Choose archetype:** default is maven-archetype-quickstart — press Enter to accept
- **Choose version:** accept the latest by pressing Enter
- **Provide coordinates:** groupId, artifactId, version, package
- **Confirm:** press Y or Enter

What Gets Generated

```
quick-start-example/
├── pom.xml                        ← coordinates pre-filled by wizard
└── src/
    ├── main/java/.../App.java     ← Hello World starter class
    └── test/java/.../AppTest.java ← starter JUnit test
```

Non-Interactive Mode

```
mvn archetype:generate \
  -DgroupId=clinic.programming.training \
  -DartifactId=quick-start-example \
  -Dversion=1.0 \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DinteractiveMode=false
```

`-D` → flag to pass any property or parameter value to Maven on the command line

`-DinteractiveMode=false` → skip all prompts and generate immediately using the provided values

Common Archetypes

Archetype	What it generates
maven-archetype-quickstart	Simple JAR with hello world and JUnit
maven-archetype-webapp	Basic Java web application (WAR)
maven-archetype-plugin	A Maven plugin project

⚠ **NOTE: Archetype templates are not always up to date.** The generated pom.xml may have outdated dependency versions (e.g. JUnit 3.8.1 instead of 4.12). Always review and update after generation.

INTERVIEW Q&A

Q: What is a Maven Archetype?
A: A project template that generates a complete project skeleton including folder structure, a starter pom.xml, and sample source code. Provided by the Maven Archetype Plugin using the archetype:generate goal.

Q: What is maven-archetype-quickstart?
A: The default Maven archetype. Generates a simple JAR project with standard Maven directory structure, pom.xml with coordinates filled, a hello world class, and a starter JUnit test.

Q: Difference between interactive and non-interactive mode?
A: Interactive mode prompts for each value one at a time. Non-interactive mode uses -D flags to pass all values directly with -DinteractiveMode=false, generating immediately without prompts.

TOPIC 16 — MAVEN WITHIN ECLIPSE

Setting Up Maven in Eclipse

- **Eclipse has built-in Maven support** via the M2E plugin
- **Configure Eclipse to use YOUR system Maven,** not Eclipse's bundled version — ensures consistency with CLI builds

```
Window → Preferences → Maven → Installations → Add
```

Point to your Maven installation folder. Check the box next to it. Click OK.

Creating a New Maven Project

```
Project Explorer → Right Click → New → Other → Maven → Maven Project
```

- **Uncheck 'Use default Workspace location'** — keep projects folder separate from Eclipse workspace
- **Select maven-archetype-quickstart,** fill in coordinates, click Finish
- **Eclipse generates the same structure** as mvn archetype:generate from the CLI

Importing an Existing Maven Project

```
Right Click → Import → Import → Maven → Existing Maven Projects
```

- **Browse to your projects folder** — Eclipse scans for pom.xml files
- **Eclipse detects projects already in workspace** and leaves them unchecked to avoid duplicates

POM Editor Tabs — Know All Five

Tab	What it shows
Overview	General project info — coordinates, parent, properties
Dependencies	All declared dependencies with scope — add/edit visually
Dependency Hierarchy	Full transitive dependency tree — graphical view, easier than CLI
Effective POM	Merged view of Super POM + parent POMs + your POM — every setting in effect
pom.xml	Raw XML source — always in sync with all other tabs

Running Maven Builds in Eclipse

```
Right Click Project → Run As → Maven install
Right Click Project → Run As → Maven clean
Right Click Project → Run As → Maven Build...   ← custom configuration
```

Build output appears in the Console window. Maven Build... lets you specify goals, JRE, and environment variables.

Eclipse Files and .gitignore

- **.classpath:** auto-regenerated from pom.xml whenever dependencies change — never commit
- **.settings/:** IDE-specific preferences — never commit
- **.project:** Eclipse project descriptor — optional, include if you want IDE auto-detection

⚠ **NOTE:** The Effective POM tab in Eclipse is equivalent to running mvn help:effective-pom from the CLI — it shows every setting Maven has in effect, including all defaults inherited from the Super POM.

INTERVIEW Q&A

Q: How do you integrate Maven with Eclipse?
A: Eclipse has built-in Maven support via M2E. Configure it to use your system Maven via Window → Preferences → Maven → Installations for consistent behaviour between IDE and CLI builds.

Q: What are the five POM editor tabs in Eclipse?
A: Overview (general info), Dependencies (manage dependencies visually), Dependency Hierarchy (full transitive tree), Effective POM (merged view of all POMs), and pom.xml (raw XML source).

Q: What is the Effective POM tab?
A: Shows the fully merged POM combining your project's POM with the Super POM and any parent POMs — every setting actually in effect. Same as running mvn help:effective-pom from CLI.

Q: What Eclipse files go in .gitignore?
A: .classpath and .settings/ — always. Both are auto-generated and IDE-specific. The .project file is optional depending on whether you want the project to be IDE-agnostic.